FH University of Applied Sciences TECHNIKUM WIEN

Structured Programming Lab

Bachelor Program Business Informatics

Department of Computer Science

Selection Statements

Conditional Statement

Overview

- some statements in a program should only be executed under certain conditions
- the conditions can be checked in an if statement

ConditionalStatementDemo - Code

```java
public class ConditionalStatementDemo {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.printf("grade: ");
        int grade = sc.nextInt();

        if (grade == 5) {
            System.out.printf("failed.\n");
        }

        System.out.printf("Thank you.");
        sc.close();
    }
}
```

ConditionalStatementDemo - Flowchart

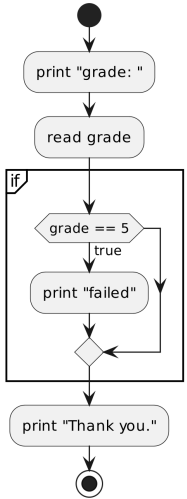


Figure 1: Flowchart ConditionalStatementDemo

ConditionalStatementDemo - Output

- failed case

```
grade: 5
failed.
Thank you.
```

- non-failed case

```
grade: 3
Thank you.
```

ConditionalStatementDemo - Summary I

- equality operator ==
  - tests for equality
  - returns `true` if the two values are equal
  - returns `false` if the two values are *not* equal
- more comparisons are <=, <, >, >=, != (see Relational Operators)

Conditional Statement - Syntax & Semantic I

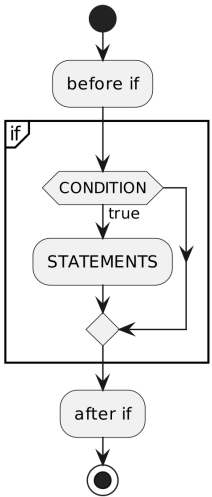


Figure 2: Flowchart

```
//before if

if (CONDITION) {
    STATEMENTS
}

//after if
```

Conditional Statement - Syntax & Semantic II

- `CONDITION` is a `boolean` expression
- `CONDITION` is always evaluated
- only if `CONDITION` is `true` the `STATEMENTS` are executed

Code Formatting

- a block opens (`{`) after `if` and closes (`}`) in the same column
- `STATEMENTS` are indented

Single Alternative

Single Alternative - Code

```
1
2  public class SingleAlternativeDemo {
3
4      public static void main(String[] args) {
5          Scanner sc = new Scanner(System.in);
6          System.out.printf("grade: ");
7          int grade = sc.nextInt();
8
9          if (grade == 5) {
10             System.out.printf("failed.\n");
11         } else {
12             System.out.printf("passed.\n");
13         }
14
15         System.out.printf("Thank you.");
16         sc.close();
17     }
18
19 }
```

Single Alternative - Console

- failed case

```
grade: 5
failed.
Thank you.
```

- passed case

```
grade: 3
passed.
Thank you.
```

Single Alternative - Summary I

- We can specify what shall happen if `CONDITION` is `false` in the `else` block

```
1 //before if/else
2
3 if (CONDITION) {
4     STATEMENTS_1
5 } else {
6     STATEMENTS_2
7 }
8
9 //after if/else
```

- Semantic:
  - `CONDITION` is always evaluated

Single Alternative - Summary II

  - only if `CONDITION` is `true` the `STATEMENTS_1` are executed
  - otherwise (if `CONDITION` is `false`) the `STATEMENTS_2` are executed

- use single alternative if a problem/system has exactly two possible cases/states, one of which must be picked/realized

Single Alternative - Flowchart

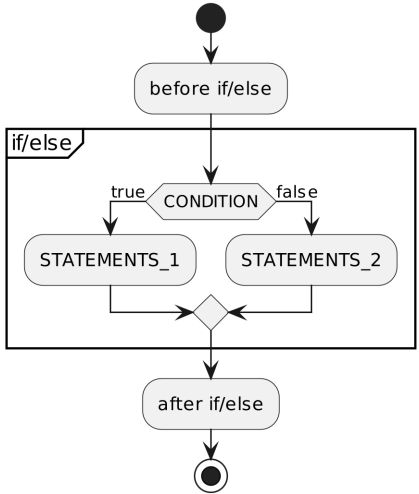


Figure 3: Flowchart SingleAlternative

Single Alternative - Code Formatting

- a block opens ({) after `if` and closes (}) in the same column
- a block opens ({) after `else` and closes (}) in the same column
- `STATEMENTS_1` and `STATEMENTS_2` are indented

Single Alternative - MiniExercise

Modify `SingleAlternativeDemo` such that the `if` case covers *passing* grades.

```
1    if (/* find the correct condition*/) {
2        System.out.printf("passed.\n");
3    } else {
4        System.out.printf("failed.\n");
5    }
```

Consecutive ifs

Consecutive ifs - Code

```java
Scanner sc = new Scanner(System.in);
System.out.printf("grade: ");
int grade = sc.nextInt();

if (grade < 6 && grade > 0) {
    System.out.printf("valid\n");
}else {
    System.out.printf("invalid\n");
}

if (grade == 5) {
    System.out.printf("failed.\n");
} else {
    System.out.printf("passed.\n");
}

System.out.printf("Thank you.");
sc.close();
```

Consecutive ifs - Console

- valid and failed case

```
grade: 5
valid
failed.
Thank you.
```

- vaild and passed case

```
grade: 3
valid
passed.
Thank you.
```

- invalid and passed case (makes no sense - will be fixed)

```
grade: 10
invalid
passed.
Thank you.
```

Consecutive ifs - Summary

- logical AND operator `&&`
  - yields `true` if and only if both sides are `true`
- other logical operators
  - OR `||` yields `true`if at least on side is `true`
  - NOT `!` "inverts" the value of its right side (`true` -> `false`, `false` -> `true`)
- Condition of second if (line 11) is evaluated, regardless of the value of condition of first if (line 5)
- In this problem, however, there is a logical connection (given by the problem and implemented in the conditions)
- Use consecutive ifs when the two conditions are independent of each other

Consecutive ifs - Flowchart

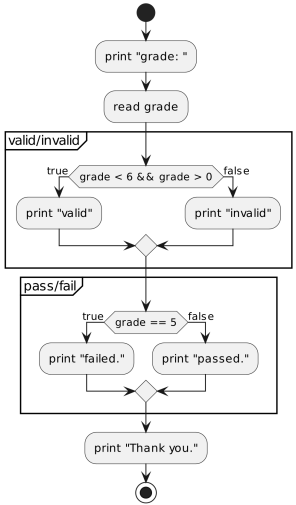


Figure 4: Flowchart Consecutive Ifs (Example)

Nested Ifs

Nested Ifs - Code

```
1         Scanner sc = new Scanner(System.in);
2         System.out.printf("grade: ");
3         int grade = sc.nextInt();
4
5         if (grade < 6 && grade > 0) {
6             System.out.printf("valid\n");
7             if (grade == 5) {
8                 System.out.printf("failed.\n");
9             } else {
10                System.out.printf("passed.\n");
11            }
12        } else {
13            System.out.printf("invalid\n");
14        }
15
16        System.out.printf("Thank you.");
17        sc.close();
```

Nested Ifs - Console

- valid and failed case

```
grade: 5
valid
failed.
Thank you.
```

- valid and passed case

```
grade: 3
valid
passed.
Thank you.
```

- invalid case

```
grade: 10
invalid
```

Nested Ifs - Summary

- outer if/else structure is the top-level distinction valid/invalid
- nested if/else structure is only executed if outer condition was `true` (we have a valid grade)
- conditions are logically connected
- full inner condition is `(grade < 6 && grade > 0) && (grade == 5)`

Nested Ifs - Flowchart

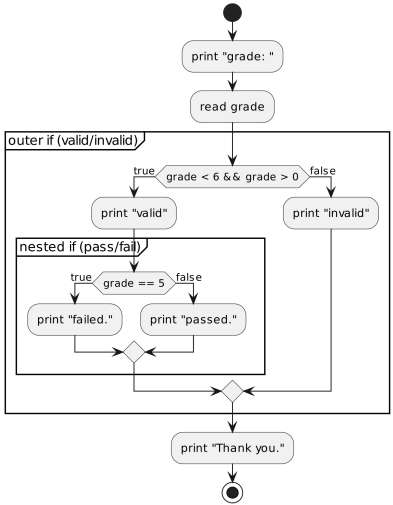


Figure 5: Flowchart Nested Ifs (Example)

Multiple Alternative

Multiple Alternative - Code

```
1         Scanner sc = new Scanner(System.in);
2         System.out.printf("grade: ");
3         int grade = sc.nextInt();
4
5         if (!(grade < 6 && grade > 0)) {
6             System.out.printf("invalid\n");
7         } else if (grade == 5) {
8             System.out.printf("failed.\n");
9         } else if (grade == 4) {
10            System.out.printf("close fail.\n");
11        } else if (grade == 3) {
12            System.out.printf("minimum pass.\n");
13        } else if (grade == 2) {
14            System.out.printf("very good.\n");
15        } else {
16            System.out.printf("excellent.\n");
17        }
18
19        System.out.printf("Thank you.");
20        sc.close();
```

Multiple Alternative - Console

- failed case

```
grade: 5
failed.
Thank you.
```

- minimum passed case

```
grade: 3
minimum pass.
Thank you.
```

- invalid case

```
grade: 10
invalid
Thank you.
```

Multiple Alternative - Summary

- Semantics
  - conditions are evaluated top to bottom
  - statements of first true condition are executed
  - and control is tranferred to the end of the *complete* structure (after `else`)
  - if no condition is true, the else branch is executed
- else is optional
- use multiple alternative **with** an else branch when a problem/system has several possible cases/states, and **exactly one** of them must be picked/realized
- use multiple alternative **without** an else branch when a problem/system has several possible cases/states, and **at most one** of them must be picked/realized

Multiple Alternative - Flowchart

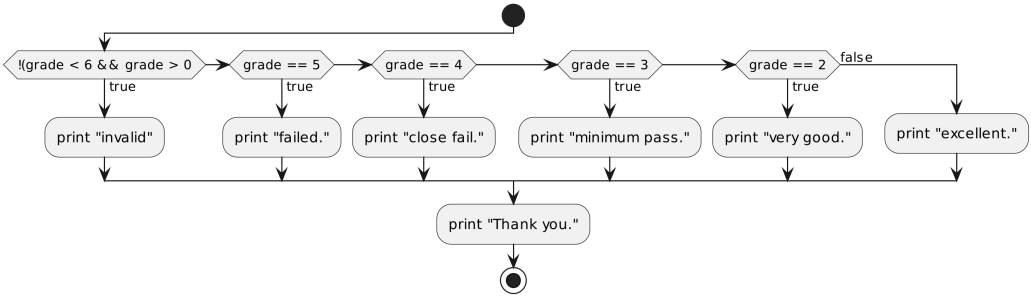


Figure 6: Flowchart Multiple Alternative (Example)

Switch/Case

Switch/Case - Overview

- a `switch` statement can be used to check different cases of an integer or a String against several constants
- it is sometimes a more readable alternative to the `if`/`else` - `if` logic

Example Simple Text Menu

```
1  Scanner sc = new Scanner(System.in);
2  System.out.println("Please enter 1-3: ");
3  int a = sc.nextInt();
4
5  switch(a) {
6      case 1:
7          System.out.println("Selection 1\n");
8          break;
9      case 2:
10         System.out.println("Selection 2\n");
11         break;
12     case 3:
13         System.out.println("Selection 3\n");
14         break;
15     default:
16         System.out.println("Wrong input.\n");
17 } /*End of switch*/
```

- the value in a will be compared with the cases
- if none of the cases is equal to the value in a the default case will be executed
- the default case is optional (it similar to “else” in the if statement)
- the “break;” statement is used to jump to the end of the switch block as soon as a case is true, if there is no break all following cases will be executed after the first one is true (this is called “fallthrough”)

Example Number Days

```java
Scanner sc = new Scanner(System.in);
int month = sc.nextInt();

switch (month) {
    case 1:
    case 3:
    case 5:
    case 7:
    case 8:
    case 10:
    case 12:
        System.out.println("31 days");
        break;
    case 4:
    case 6:
    case 9:
    case 11:
        System.out.println("30 days");
        break;
    case 2:
        System.out.println("28 days (or 29 if leap year)");
        break;
    default:
        System.out.println("Invalid month.");
}
```

Relational and Conditional Operators

Relational and Conditional Operators - Overview

- `if` tests a condition
- the condition must be a `boolean` expression
- **relational operators** like < (less than) compare two (numeric) values and return a boolean value
- **conditional** operators like || (logical OR) operate on boolean values and return a boolean value
- with the types of operators we can construct logical expressions and conditions

Relational Operators

Relational Operators compare two (numeric) expressions and return a `boolean` value.

Table 1: Relational Operators

Symbol	Operation
<	Less Than
<=	Less Than Or Equal
>	Greater Than
>=	Greater Than Or Equal
==	Equal
!=	Not Equal

Conditional Operators I

Conditional Operators operate on `boolean` expressions and return a `boolean` value.

Table 2: Conditional Operators

Symbol	Operation
`&&`	AND
`||`	OR
`!`	NOT (unary operator)

Conditional Operators II

Table 3: Truth Table (OR, AND, NOT)

A	B	A || B	A && B	!A
true	true	true	true	false
true	false	true	false	false
false	true	true	false	true
false	false	false	false	true